DAY 40

Testing Machine Learning APIs with Python

A practical guide to Flask test clients, pytest, assertions, and input validation

Learning outcome: By the end of this guide, you will be able to write repeatable tests for a prediction API and understand what those tests can and cannot prove.

1. Why do we test an ML API?

A machine-learning API accepts a request, validates the input, sends valid data to a model, and returns a response. If any part fails, users may receive confusing errors or incorrect response formats. Testing helps us catch these problems before deployment.

Examples of what to test

• A health endpoint returns HTTP 200 and a clear status message.

• Valid input returns a JSON response containing a prediction.

• Missing fields return a helpful HTTP 400 error.

• Out-of-range values, such as attendance = 140, are rejected.

• The response stays consistent and does not unexpectedly become an HTML error page.

2. Key terms

Testing: Checking whether software behaves as expected.

Automated test: A test that runs through code instead of relying on manual checking.

pytest: A Python testing framework that discovers functions whose names start with test_.

assert: A statement that fails the test if a condition is false.

Test client: Flask's test_client() sends simulated HTTP requests directly to the app without starting a public server.

Status code: An HTTP number describing the result, such as 200 for success and 400 for a bad request.

Fake model: A predictable stand-in used to test API behavior without loading a real trained model.

3. Install and run

Install the required libraries in your Python environment:

```
python -m pip install flask pytest
```

Save the practice file and run:

```
pytest -v Day-40-Flask-API-Testing-Practice.py
```

The -v option means verbose. It displays each test name and whether it passed.

4. What the sample API does

The example app has two endpoints:

GET /health returns a small JSON response showing the service is responding. **POST /predict** expects JSON with study_hours and attendance, validates both values, and returns a prediction from a FakeModel.

The FakeModel calculates a demonstration score using study_hours × 10 + attendance × 0.3. This formula is only for predictable testing practice. It is not a trained ML model and must not be treated as a real student-performance predictor.

5. Understanding a test

```
def test_health_endpoint():
    client = create_app().test_client()
    response = client.get("/health")

    assert response.status_code == 200
    assert response.get_json() == {"status": "ok"}
```

Line by line:

• **def test_health_endpoint():** defines a test. pytest recognizes the test_ prefix.

• **create_app().test_client()** creates a test client for the Flask application.

• **client.get('/health')** simulates a GET request.

• **response.status_code == 200** checks that the endpoint reports success.

• **response.get_json()** reads the JSON response body.

• **assert** makes pytest mark the test as failed if the expected condition is not true.

6. Test cases in this practice file

Health endpoint
Input: GET /health
Expected: 200 and status=ok

Valid prediction
Input: study_hours=4, attendance=80
Expected: 200 and numeric prediction 64.0

Missing field
Input: attendance omitted
Expected: 400 and missing_fields identifies attendance

Invalid attendance
Input: attendance=140
Expected: 400 with an attendance validation error

Wrong data type
Input: study_hours='four'
Expected: 400 with a numeric-input error

Hours outside range
Input: study_hours=25
Expected: 400 with a study_hours validation error

Wrong JSON shape
Input: JSON list instead of object
Expected: 400 with a clear error

7. Why use a FakeModel?

A real model may produce slightly different predictions after retraining, and loading a model can make tests slower or dependent on extra files. A fake model produces a known answer, allowing us to focus on API logic such as validation, response structure, and status codes.

When connecting a real trained model, keep API behavior tests and model evaluation separate. Use model metrics such as MAE, RMSE, accuracy, precision, or recall only when they are appropriate for the task.

8. Common mistakes

• Using the wrong test name: pytest usually discovers functions starting with test_.

• Checking only the status code but ignoring the JSON response body.

• Testing only valid input and forgetting invalid or missing values.

• Assuming a passing API test proves the model is accurate.

• Using real external services in unit tests when a deterministic fake is enough.

• Allowing invalid input to reach the model without validation.

9. Practice tasks with solutions

Task 1: Why is attendance=140 invalid in this example?

Solution: Attendance is represented as a percentage and the API allows values from 0 to 100. A value of 140 is outside the defined range, so the API should return HTTP 400.

Task 2: What is the purpose of assert response.status_code == 200?

Solution: It checks that the request returned the expected success status. If the response has another status code, pytest fails the test.

Task 3: Does a passing FakeModel test prove a real model is accurate?

Solution: No. It proves that the tested API path behaves as expected with the fake model. Real model quality must be measured separately on suitable evaluation data.

Task 4: Add a test for negative attendance.

```
def test_negative_attendance_returns_400():
    client = create_app().test_client()
    response = client.post(
        "/predict",
        json={"study_hours": 4, "attendance": -1},
    )
    assert response.status_code == 400
```

10. Day 40 checklist

■ I understand what pytest does and how assert works.

■ I can create a Flask test client.

■ I can test an endpoint's status code and JSON response.

■ I can test valid input, missing fields, wrong types, and out-of-range values.

■ I understand that API behavior tests do not measure model accuracy.

Quick revision

pytest discovers test functions, test_client() simulates HTTP requests, assert checks expectations, status codes describe request outcomes, and a FakeModel makes API tests predictable. Strong API tests include both successful and unsuccessful requests.

Learning repository: https://github.com/DeveloperDesigner-star/Datasciencepdfs